Mycelia — Full Scripting Order Request

This document describes the Mycelia Roblox game and the full scope of scripting work I need built. Mycelia is a cozy mushroom-cultivation game with hidden depth — Bee Swarm Simulator's longevity model wrapped in Stardew-Valley-style aesthetic. The Pre-Alpha core loop already works (harvest → plant → sell → brew); this doc describes everything needed to take it from Pre-Alpha to a complete launchable game.

Scripter scope: gameplay code (server + client logic, validation, persistence), UI behavior wiring (button handlers, state binding, animations, transitions), animation scripting (playing supplied animations on triggers), sound scripting (playing supplied audio on triggers, spatial audio), camera scripting, and bug fixes.

What I will supply (NOT scripter scope): 3D models (mushrooms, NPCs, props), UI visual designs and assets (button textures, panel art, fonts), animations (character + NPC rigs animated separately), audio files (music, SFX), and VFX particle definitions (emitter properties + texture assets). The scripter integrates these into working systems — does not author them.

There is no combat, no PvP, no death penalty in Mycelia. Skip any combat-related patterns from previous projects. The game is cozy, social, and economy-driven.

Project context

Project root: C:\Users\fonte\Documents\Mycelia\mycelia. Uses Rojo (7.7.0-rc.1, pinned via aftman.toml) for live syncing from VS Code into Studio. Server-authoritative validation is the established pattern — clients send intents via remotes, server re-validates everything before mutating state. The codebase has 74 unit tests that auto-run on Play in Studio via a custom TestKit framework (see Tests/RunTests.server.lua).

The place is published to Roblox under the name "Mycelia." DataStore saves are enabled.

Key architectural files to read first: HANDOFF.md (full project state), src/ServerScriptService/Main.server.lua (server entry point + startup order), src/ReplicatedStorage/Shared/Constants.lua (every tunable in the game).

What's already built (do not rewrite)

- Core gameplay loop: walk, harvest wild mushrooms, plant on plots, wait for ripen, harvest planted, sell at witch.
- Brewing foundation: cauldron interaction, 4 methods (Steep/Boil/Ferment/Dry-grind), recipe lookup table, 6 starter recipes.
- 3 active potion effects: Common Speed Potion (walkspeed buff), Hearth Brew (plot growth buff), Dewdrop Tonic (wild yield buff). Potion consume flow with HUD chips.
- 15 mushroom species across 4 tiers (Common/Uncommon/Rare/Epic). Save schema v2 supports persistence.
- Brewing Journal UI showing discovered + undiscovered potions.

- Painted terrain (mossy mountain with cave interior, pond, paths, hills, dirt patches). Lighting + atmosphere setup.
- Animated koi fish swimming in the pond.
- Inventory + selling + brewing math, all server-validated.
- DataStore persistence with autosave + graceful degradation when DataStore unavailable.
- TestKit + 74 unit tests (Inventory, Selling, Species, PotionEffects, Recipes).
- Hand-built escape valves: every procedurally-built piece (ground, atmosphere, trees, witch, cauldron, plots) can be replaced with a hand-placed instance via documented attribute/name contracts.

Game vision (what we're building toward)

Players walk into a misty forest, harvest glowing mushrooms with a tap, sell them to the Forest Witch, plant their own patches with the coins, and discover that mushrooms can be brewed into potions with combinatorial recipes. Over time players unlock new biomes, attract Forest Spirit pets, run shops at the trading post, and join co-op foraging expeditions. The depth layer (substrate optimization, recipe discovery, season cycles) gives veterans 200+ hours of meaningful play. Mobile-first, no level cap, no pay-to-win. Monetization is time and cosmetics only.

Full design doc: C:\Users\fonte\Documents\Mycelia\mushroom_forest_design_doc.md. That doc is the north star — refer to it when this scripting order is ambiguous.

System: Mushroom Pokedex (collection)

Launch target is 60 species across 6 rarity tiers (Pre-Alpha currently has 15 across 4 tiers). Each species has a unique id (used as save key — never rename), display name, tier, base sell price, growth time in seconds, and a brewingTags table (currently empty placeholder; populates as the brewing depth layer rolls out).

Tiers and how they're obtained:

Tier	Source	Drop weight	Sell mult
Common	Wild spawn anywhere	100	1×
Uncommon	Cultivation w/ specific substrate	35	2×
Rare	Biome-specific + conditions	10	5×
Epic	Weather + season + biome	2	12×
Legendary	Lunar cycle + recipe discovery	0.3	35×
Mythic	Community-discovered events	0	100×

Each player has a discovered set: { [speciesId] = true } stored in stats.speciesDiscovered. Once a player holds a species in their inventory once (wild harvest, plot harvest, trade, or quest reward), that species is permanently flagged as discovered. The Pokedex UI is the browseable catalog of discovered + undiscovered species.

Pokedex UI behavior

1. Player taps the Pokedex button on the HUD.
2. A panel opens listing all 60 species. Top of panel shows "Discovered: X / 60" counter.
3. Filter buttons at the top: All / Common / Uncommon / Rare / Epic / Legendary / Mythic / Seasonal.
4. Each row: species thumbnail, name, tier color dot, brief description.
5. Discovered species: full details visible. Tapping a row opens a detail card with thumbnail, full description, sell price, where to find (biome / substrate hints), known recipes that use it.
6. Undiscovered species: name shows as "???", thumbnail shows as silhouette, description shows "Not yet discovered." Tier color dot still visible (so players can see what's missing per tier).
7. Sort options: by tier, by name, by recently discovered, by sell price.

System: Wild mushroom spawning

Each biome has a wild-spawn area (an invisible BasePart in Workspace, named WildSpawnArea per biome). The spawner periodically places mushroom Models inside the volume up to a per-biome cap. Wild mushrooms can be harvested by any player.

Spawn behavior:

1. Server runs a per-biome spawn loop. Every N seconds (default 8), if active wild mushroom count < cap, spawn one.

2. Spawn picks a random species from the biome's wild-spawn table (weighted by tier dropWeight, modified by current season + lunar phase).

3. Pick a random point inside the WildSpawnArea volume. Drop the mushroom Model on the surface (raycast down).

4. Each wild mushroom Model has attributes: isWildMushroom=true, speciesId, biomeId. A ProximityPrompt named WildHarvestPrompt is parented inside the cap Part.

5. When a player triggers the prompt (E on PC, tap on mobile), server validates and adds 1 of that species to the player's inventory (multiplied by any active wildYield potion bonus).

6. Mushroom Model is destroyed. Server schedules a respawn 0.5 seconds later to prevent the area from feeling empty.

Spawn rates, caps, and per-biome wild tables are all in Constants.lua. Per-species visual variants (cap color, glow, spots) are defined in WildSpawn.lua's SPECIES_VISUALS table — for new species, scripter adds a row matching the existing pattern.

System: Cultivation (the depth layer)

Players plant mushrooms on plots they own. The plant-and-wait mechanic is the surface level; underneath sits the variable system that makes Mycelia a 200-hour optimization puzzle (the Bee Swarm parallel). Pre-Alpha currently ships only the surface layer (4 shared plots, single-variable cost+time). The full system needs all six variables.

Plots

Each player owns N plots (default 4 from spawn, +N from gamepass purchases). Plots are physical Models in the world, owned by the player (server tracks ownership via plot.plantedByUserId attribute, currently used; extend to plot.ownerUserId for permanent ownership).

Plot states: Empty → Growing → Ripe → Empty (after harvest). Each plot has:

- A Soil base (BasePart) with attribute isPlot=true.
- Three child Folders: EmptyVisual / GrowingVisual / RipeVisual. Visibility toggled by Planting based on state.
- Two ProximityPrompts: PlantPrompt (enabled when Empty) and HarvestPrompt (enabled when Ripe).
- Attributes set on plant: state, speciesId, plantedAt, ripeAt, plantedByUserId, substrateType, moisture, light, hostTreeId, soilPH.

Plant flow

1. Player walks to an empty plot they own. PlantPrompt appears.

2. Press E. Client opens the planting UI: a card listing every species the player has discovered and can afford.

3. For each species in the list: name, tier color, plant cost (in coins), required substrate, expected growth time, and a small "info" button that opens the species detail card.

4. Filter buttons at the top: All / by tier / by substrate.

5. Player taps a species. If the plot has a non-default substrate already loaded, the UI highlights compatibility (green = grows here, yellow = grows but reduced yield, red = will not grow). If the plot's substrate is wrong, player can opt to change it (consumes a substrate item from inventory, costs nothing else).

6. Player taps Plant. Client sends PlantSpecies remote with (plot, speciesId, substrate, additives).

7. Server re-validates everything: plot is empty + owned by player, species discovered, player has cost + substrate + additives in inventory. Returns failure code if any check fails.

8. On success: server deducts coins + substrate + additives, sets plot state to Growing, computes ripeAt = now + species.growthTimeSeconds × CULTIVATION.growthTimeMultiplier × player's potion multipliers × substrate compatibility multiplier.

9. Plot's GrowingVisual becomes visible. PlantPrompt hidden.

10. Optional: play a brief plant animation on the player character (scripter wires up the supplied animation; user supplies the animation asset).

Cultivation variables (full depth, post-Pre-Alpha)

Six variables affect what grows + how well:

• Substrate type — compost, hardwood, straw, dung, peat, magical loam. Each species has a preferred substrate; planting on the wrong one yields fewer mushrooms or fails entirely. Substrate items are inventory items obtained via quests/shops. Unlocked over time as the player progresses.

• Moisture level — set by watering action (player walks to plot, presses Water prompt). Decays over time. Affected by weather (rain raises it). Different species prefer different moisture ranges.

• Light level — sunny / dappled / dark. Determined by plot location in the world (some plots near tall trees are dappled, plots in caves are dark). Player chooses where to plant their plots.

• Host tree — some species require a specific tree species nearby (within X studs). The tree types live in Workspace as tagged Models.

• Soil pH — 0–14 scale. Adjusted with additives (acid/base items from inventory). Different species prefer different pH ranges.

• Companion mushrooms — some species growing in adjacent plots boost each other. Some inhibit. Lookup table in Constants.lua.

Each variable's effect is computed at plant time and locked in for that plot's growth cycle (changing moisture mid-growth doesn't retroactively change the outcome). Yield (number of mushrooms harvested) and rare-variant chance are both modified by the variable match quality.

Harvest a ripe plot

1. Player walks to ripe plot. HarvestPrompt appears.
2. Press E. Server computes yield based on substrate match + variable match quality.
3. Server adds N of that species to inventory, plus rare-variant chance check (low % chance to receive an Uncommon-tier variant of a Common species, or Rare-variant of an Uncommon, etc.).
4. Plot resets to Empty state. Substrate is consumed unless the player has the "Recyclable Substrate" gamepass.
5. Optional harvest animation + spore puff VFX (scripter wires up; user supplies assets).

System: Inventory

Each player has an inventory split into categories. Categories are: Mushrooms (raw), Mushrooms (dried), Potions, Spores, Substrates, Additives, Recipes (scrolls), Forest Spirit Items, Quest Items, Cosmetics, Tools.

Each item type has the following attributes:

• category (enum)

• stackable (bool) — if true, multiple instances stack into one slot with a count badge. Max stack 999.

• tradable (bool) — if true, item can be sold to NPCs or traded to other players.

• droppable (bool) — if true, dropping the item drops it to the ground as a pickup; false means it disappears on drop.

• iconAssetId (Roblox image asset for the inventory thumbnail)

Backpack UI behavior (you supply the visual design):

1. Player taps the Backpack button on the HUD.
2. Backpack panel opens. Tabs at the top for each category. Default tab: Mushrooms.
3. Each tab shows a grid of slots (4×8 = 32 slots per category by default; expandable via gamepass).
4. Hover/short-tap on a slot: tooltip shows item name + description + stats.

5. Long-press on a slot: action menu appears — Drop, Use (if applicable), Trade (if in trading mode), Inspect.

6. Drop confirmation prompts before actually dropping (prevents misclicks losing rare items).

7. Stackable items show a count badge in the corner of the icon.

8. Empty slots are visible (greyed out).

Inventory state lives in PlayerData, persists via DataStore. Every mutation goes through server validation. Inventory.lua already has the pure math (Inventory.add, Inventory.remove, Inventory.count) — extend the existing module rather than rewriting.

System: Selling (Forest Witch + other merchants)

Multiple NPC merchants exist. Each merchant has a list of item categories they buy + a list of items they sell. Merchant interaction is via ProximityPrompt → opens a shop UI.

Forest Witch (the primary merchant)

Lives in the cave chamber inside the mountain east of spawn. Buys all mushrooms (raw and dried). Sells substrates, additives, spores, and basic potions for coins.

Sell flow (existing — already implemented):

1. Player walks up to witch. SellPrompt appears.

2. Press E. Currently this auto-sells the entire inventory of mushrooms and shows a "+N coins" toast. We want to upgrade this to a proper shop UI per below.

Upgraded shop UI (to be built):

1. Player presses SellPrompt. Shop UI opens, two tabs: Sell and Buy.

2. Sell tab: lists every item in the player's inventory the witch will buy, with per-item sell price (= species.baseSellPrice × tier.sellMultiplier × witchPriceMultiplier). Each row has a +/- quantity selector and a checkbox to include in the sell batch.

3. "Sell All" and "Sell Selected" buttons at the bottom. Total coin payout displayed.

4. Buy tab: lists items the witch sells (substrates, additives, spores). Each row: name, price, +/- quantity selector, Buy button.

5. Player's current coin balance + inventory space remaining shown at the top.

6. Server validates every transaction: player has the items / coins, inventory has space.

Other merchants (to be built)

• Substrate Merchant — sells substrate types as they unlock. Located in Starter Glade after the first quest.

• Spore Merchant — sells species spores (alternative to wild discovery for filling Pokedex). Some species available, others have to be traded for or harvested.

- Spirit Food Merchant — sells items that increase Forest Spirit attraction chance.
- Cosmetic Merchant — sells character outfits, hat items, particle trails. Coins or Robux.
- Decoration Merchant — sells house/hut decorations.

All merchants share the same shop UI script. Each merchant has a config table specifying which categories they buy + which items they sell + per-item price modifiers.

System: Brewing + cauldron

The cauldron is the secret-weapon depth system. Mushrooms are ingredients; combinations produce potions. The Pre-Alpha brewing system already works for the basic case (4 methods, 1–5 ingredients per brew, 6 known recipes). Full system needs the recipe space to scale (thousands of possible combinations) and effects to reach the design-doc categories.

Brewing UI (already built — minor enhancements)

Existing UI is functional: method buttons row, scrolling ingredient list from inventory, selected summary, Brew/Cancel buttons. Enhancements needed:

- Add a small "recipe attempts" counter — how many distinct combinations the player has tried (encourages experimentation).
- Add a hint button next to a recipe-mystery icon: opens a modal with discovery hints (e.g., "Players have brewed 3-Common-Cap recipes 142 times this server. 8 of them produced potions."). Server-collected stats, gives players signals about productive directions without spoiling specific recipes.
- Add a "recent brews" history (last 10 attempts by the player) above the method row.

Recipe lookup

Recipes are matched by (method + sorted ingredient ids) → potion id. Failed brews intentionally consume ingredients (engagement design — experimentation has cost). All recipes are server-side; never leak to client.

Recipe space at full scope: 60 species × 4 methods × 1–5 ingredient combos = thousands of combinations. Most are duds (fizzle). Some produce potions. Discovery is the long-term engagement loop. Add new recipes by adding a single line to Recipes.lua's recipe(...) list.

Potion catalog

Potions are categorized:

- Buffs — speed, harvest yield, luck (rare drop chance), drying time. Apply timed effects.
- Tools — see hidden mushrooms, reveal map areas, attract spirits. Modify world perception.
- Cosmetic — temporary glow effects, color changes, particle trails. Tradeable status items.
- Forest interaction — rain summons, day/night control on private plots, season shifts.

Pre-Alpha ships 6 potions, 3 with effects. Full launch target is 30+ potions across all four categories. Each potion has an entry in Constants.POTIONS specifying its kind and effect parameters; PotionEffects.lua handles application.

System: Potion effects + consume

Existing PotionEffects.lua handles 3 effect kinds (speed, growth, wildYield). Extend to support all four design-doc categories. Effect kinds:

- speed — multiplies Humanoid.WalkSpeed for N seconds.
- growth — applies a growth-time multiplier to the player's plots (one-shot acceleration on currently-growing + multiplier on new plants for duration).
- wildYield — wild harvests award (1 + bonus) for duration.
- luck — increases rare-variant drop chance for duration.
- harvestYield — planted harvests yield more for duration.
- reveal — within X studs of the player, hidden mushrooms (a new visibility flag on wild spawns) become visible for duration.
- spiritAttract — increases spirit attraction chance for duration.
- cosmeticGlow — gives the player character a colored particle trail for duration.
- weatherSummon — summons rain on the local biome for N minutes (see Weather system).
- timeShift — allows the player to set day/night on their own plots (toggle UI button while active).

Consume flow (existing pattern, extend for new kinds):

1. Player taps the Potions HUD button. Potion picker opens.
2. Each potion row: name, owned count, effect summary, tappable if active (has a Constants.POTIONS entry), greyed out if inert.
3. Player taps an active potion. Client fires ConsumePotion remote.
4. Server validates inventory, deducts 1, applies effect, fires ActiveEffectsUpdated remote.
5. Client renders an active-effect chip below the Potions button (one chip per kind, color-coded). Live countdown.
6. On expiry: server fires ActiveEffectsUpdated without the expired entry. Client removes the chip.

Stacking rule: same-kind potions consumed during active duration refresh the timer but do not double the magnitude. Fresh-vs-refresh distinction matters for one-shot effects like growth (already handled in PotionEffects.lua).

System: Brewing Journal (Pokedex for potions)

Already built (JournalController.client.lua). Lists all potions in the catalog. Discovered ones show full details; undiscovered show as "???". Counter at top: "3 of 6 discovered." Sorted: discovered first, then by rarity, then by name.

Enhancements for full launch:

- Add ingredient hints for discovered potions (which species + method combo produced it). Toggleable in Settings — some players prefer not to be reminded.
- Add owned count next to each discovered potion.
- Add a search bar to filter by name.
- Add a category filter (Buffs / Tools / Cosmetic / Forest interaction).

System: Forest Spirits (pets + companions)

The pet/companion layer. Specific mushrooms attract specific spirits when held in inventory or planted in plots. Spirits are tradeable Models that wander the player's plot area and grant passive bonuses.

Spirit attraction

1. Each spirit type has an attraction map: { [speciesId] = baseChancePerHour }.
2. Server runs a per-player spirit-attraction tick every N minutes (default 5). For each player, sum baseChancePerHour for every species currently in their inventory or growing on their plots, multiplied by any active spiritAttract potion bonus.
3. Roll the chance. On hit, spawn a spirit Model near the player's plot area (or in their hut if defined). Player gets a notification: "A Mossling has wandered onto your plot!"
4. Player can claim the spirit by walking up to it and pressing E. Claimed spirits go into the player's spirit roster.
5. Unclaimed spirits despawn after 5 minutes.

Spirit roster

Player has a max active roster (default 3, expandable via gamepass). Each spirit in the active roster grants a passive effect (faster growth on that plot, yield bonus, weather shift, etc. — defined per spirit type in Constants.SPIRITS).

Spirits spawned in the world (active-roster ones) are anchored Models that wander within a small radius of the player's hut, idle-animated. Scripter handles the wandering AI (simple random-target pathing within the radius). User supplies the spirit Models + wander/idle animations.

Spirit catalog (launch target)

- Common spirits: Mossling, Spritefly, Dewdrop. Each gives a small passive bonus (5–10%) on a single stat.
- Rare spirits: Lanternfox, Crystalmoth, Deerlet. Each gives a moderate bonus (15–25%) plus a unique mechanic (Lanternfox lights up dark plots, Deerlet grants a daily luck buff, etc.).
- Legendary spirits: Forest Mother (once-per-server lunar event spawn). Grants global server-wide bonuses while present.

Spirit trading

Spirits are tradeable items in the inventory. Trading a spirit removes it from your active roster and packages it as an inventory item. The receiving player can then claim it into their roster. Trade flow uses the standard trading post (see below).

System: Biomes + travel

Players start in the Starter Glade. They unlock new biomes over time. Each biome has its own substrate types, weather patterns, exclusive species, and visual aesthetic.

Launch biomes

- Starter Glade — Pre-Alpha biome. Always accessible.
- Misty Hollow — rainforest aesthetic, water-loving species. Unlock at reputation 100.
- Frostroot Pass — snowy, slow-growing high-value species. Unlock at reputation 300.
- Sunken Grove — swamp, alchemy ingredients. Unlock at reputation 600.
- Old Growth — ancient forest with legendary species. Unlock at reputation 1500.
- Glimmerwood — bioluminescent, only at night. Unlock at reputation 2500 + lunar quest.
- Lost Cathedral — ruined temple, mythic spawns during events. Unlock by community-event participation.

Travel mechanic

Each biome is a separate Place (Roblox subplace) within the Mycelia universe game. Travel uses TeleportService.

1. Player walks to the Travel NPC at spawn (or in a biome). Talks to NPC.
2. UI opens listing all unlocked biomes with thumbnails + brief description + current weather/season indicator.
3. Locked biomes shown greyed out with their unlock requirement.
4. Player taps a biome. Confirmation prompt: "Travel to Misty Hollow? Your inventory comes with you."
5. On confirm: TeleportService:Teleport to the biome's Place ID. Player loads in at the biome's spawn point.
6. All save data persists across biomes (single PlayerData per universe).

System: Foraging Expeditions (co-op)

Co-op runs into deeper biomes for high reward. 2–4 players. Time-limited. Slightly dangerous — failed expeditions lose unfinished potions. The social hook driving algorithm-friendly play.

Expedition flow

1. Player walks to the Expedition Coordinator NPC in any biome. Talks to NPC.
2. UI opens with two tabs: Create and Join.
3. Create tab: list of available expedition types (each has biome, difficulty, time limit, party size, rewards). Player selects one and gets a private lobby.

4. Lobby UI: lists current party members. Owner (creator) can kick. Optional password field for private lobbies. Min level requirement field.

5. Join tab: lists open expedition lobbies. Each row shows expedition name, current members / max, owner name, level requirement. Join button.

6. When the lobby reaches required size, owner clicks Start. Countdown begins. After countdown, all party members are TeleportService'd into the expedition Place (a separate game instance).

7. Inside the expedition: same gameplay loop (harvest mushrooms) but on a time limit. Some species only appear in expeditions. Some are bounded by environmental hazards (steep cliffs, dark areas).

8. When time runs out OR all players choose to extract: TeleportService back to the original biome. Mushrooms + drops collected during the expedition come with the players. Unfinished brewing items are lost on failure.

9. Rewards distributed based on participation (who collected what, who completed expedition objectives).

Lobby chat: party members can text-chat in a private channel during lobby and expedition. Invitation: party owner can invite friends from the friend list (if any are in the same Roblox universe).

System: Seasons + lunar cycles

Mycelia has a real-time season cycle: 1 season = 1 real-world week. 4 seasons = 4 weeks = 1 game year. Lunar cycle: real-world month. Different mushrooms fruit in different seasons; some legendary species only appear in specific moon phases.

Server-side time engine

• Server reads os.time() at startup. Computes current season + lunar phase from a fixed epoch (e.g., 2026-01-01) — this means all servers share the same season + moon phase regardless of when the server started.

• A SeasonChanged BindableEvent fires when the season ticks over. Subscribers: WildSpawn (rotates wild table), Weather (changes weather patterns), Lighting (adjusts color palette).

• A LunarPhaseChanged BindableEvent fires when the moon phase ticks over. Subscribers: WildSpawn (legendary spawns), Spirits (Forest Mother triggers).

• Replicated to clients via a SeasonState remote on player join + on changes. Client uses for UI display (season indicator on HUD).

Visual + gameplay effects per season

• Spring — pastel colors, light rain, growth time bonus on commons.

• Summer — saturated colors, sunny, harvest yield bonus on commons.

- Autumn — orange/red palette, leaves falling (VFX), legendary autumn-only species can spawn.
- Winter — desaturated, snow (VFX), Frostroot Pass biome has bonus spawns.

Lunar cycle: Full Moon enables Glimmerwood lunar mechanics (legendary spawns visible). New Moon enables stealth mechanic (player movement is quieter, attracts more wild spawns). Quarter Moons are neutral.

System: Weather

Each biome has a weather state: Sunny / Cloudy / Rainy / Stormy / Snowy / Foggy. Weather affects moisture (rain raises plot moisture), spawn chances (some species favor rain), and visuals.

1. Server runs a per-biome weather tick every 10 minutes. Probabilistic transitions based on season (e.g., autumn has more rain).
2. Current weather replicated to clients. Each client renders weather VFX (rain particles, snow particles, fog Atmosphere settings — user supplies the VFX assets, scripter wires them in).
3. Plots automatically gain moisture during rain.
4. Wild spawn loop checks weather + species weather preference when picking the next spawn.

Weather can be summoned by certain potions (weatherSummon kind) — this overrides natural weather for the local biome for the potion's duration.

System: NPCs + dialogue

Major NPCs at launch:

- Forest Witch — primary merchant, tutorial guide. Lives in the cave chamber.
- Old Hermit — gives mid-game quests, offers cultivation tips. Lives in Misty Hollow.
- Wandering Alchemist — gives brewing-focused quests, sells rare additives. Travels between biomes (changes location each in-game day).
- Spirit Speaker — gives spirit-attraction quests, sells spirit food. Lives in the Old Growth biome.
- Travel Coordinator — biome travel.
- Expedition Coordinator — co-op expedition setup.
- Substrate Merchant, Spore Merchant, Spirit Food Merchant, Cosmetic Merchant, Decoration Merchant — single-purpose shop NPCs.
- Trading Post Manager — explains the trading post to first-time visitors.
- Gardener Coach — tutorial NPC at spawn, gives the first 5 quests in sequence.

Each NPC has:

- A Model in the world (you supply).
- An idle animation (you supply).
- A ProximityPrompt with action text "Talk".
- A dialogue table — branching conversation tree with options. Defined in NPCs/.lua.
- Optional: a quest table — quests they can give (see Quest system).
- Optional: a shop config — what they buy/sell (see Shop system).

Dialogue UI

1. Player presses Talk. Dialogue UI opens at the bottom of the screen (chat-bubble style with portrait of the NPC).
2. NPC says a line. Player taps Continue or picks a response option (if multiple).
3. Some lines trigger side effects: open a shop UI, accept a quest, complete a quest, give an item.
4. Player can close the dialogue at any time (X button or Esc). Some lines lock out the close until the player picks a response.

System: Quests

Quests are the onboarding scaffold + ongoing engagement layer. NPCs offer them; players accept, complete objectives, and turn them in for rewards (recipes, biome unlocks, rare seeds, reputation, coins).

Quest types

- Tutorial quests — first 5–10 quests teach core mechanics (harvest 3 mushrooms, plant your first patch, brew your first potion, etc.). Given by Gardener Coach NPC at spawn.
- Daily quests — randomized lightweight tasks (harvest 20 mushrooms, sell 100 coins worth, brew anything new). Refresh every real-world day at midnight UTC. Reward small coin/reputation bonus.
- NPC story quests — multi-step storylines tied to specific NPCs. Unlock as the player gains reputation with that NPC. Reward recipes, biome unlocks, unique cosmetics.
- Event quests — limited-time during LiveOps events. Reward exclusive cosmetics or limited-edition spirit food.

Quest data structure

- id (unique string)
- title, description, NPC giver

• objectives — list of conditions: harvest N of species X, sell N coins, brew potion P, plant in biome B, etc.

• rewards — coins, reputation gain, items, recipes, biome unlock flags.

• prerequisites — must have completed quest X first; must have reputation ≥ N with NPC.

• repeatable (bool) — daily quests are; story quests aren't.

Quest UI

1. Player taps the Quest button on the HUD.

2. Quest journal opens with three tabs: Active / Completed / Available.

3. Active tab: each quest shows title, NPC, current objective progress (e.g. "3 / 5 BrownCaps harvested"), reward preview.

4. Tapping an active quest opens a detail panel with full objective list and a "Track" toggle (sets this quest as the HUD-tracked one).

5. Tracked quest objective is shown on the HUD in a small pinned widget (top-right by default, mobile-positionable).

6. Available tab: list of quests offered by NPCs the player can currently access. Tapping shows which NPC to talk to.

7. Completed tab: list of finished quests. Read-only history.

System: Trading Post (player-to-player)

The longevity engine. A physical place in the world (not just a menu) where players gather to trade. Located in the central hub biome (Starter Glade, near spawn). All biomes have a Trading Portal NPC that fast-travels you back to the trading post.

Trade initiation

1. Player walks up to another player inside the Trading Post area. Tap the player's character to open the player-actions menu.

2. Menu shows: "Send trade request" (enabled if not already in trade), "Visit plot" (if plot visiting unlocked), "Add friend".

3. Player taps "Send trade request". A request prompt appears for the other player: "Player X wants to trade. Accept / Decline." 20-second cooldown after a sent request.

4. On accept: trading UI opens for both players simultaneously.

Trading UI

Both players see the same UI layout:

• Left column: my backpack (categorized).

• Center: my offering area — 2×5 item slots + a coin offering input box. Below: their offering area — 2×5 item slots + their coin offering (read-only).

• City tax indicator showing the % cut taken on coin trades (default 5%, adjustable in Constants).

• Right column: their backpack (read-only — I can see what they have but not pick from it).

• Bottom: Offer / Accept / Deny buttons.

Trade flow

1. Each player drags items from their backpack into their offering slots (or taps to move). Sets coin offering.

2. Each player clicks Offer when ready. Their button becomes "Waiting…" until both have offered.

3. Once both have offered, both players see each other's offers. The Offer button becomes Accept.

4. After clicking Accept, button becomes "Waiting…" until the other accepts. Once both accept, the trade executes server-side.

5. Server validates: both players still have the offered items + coins, both players still in trading range, neither has logged out. If valid, swap items + coins.

6. Show a top-screen toast: "Trade completed." Close the trading UI.

7. If at any point: distance > effective range, a player goes out of the trading post area, a player declines, or the connection drops — trade cancels with a "Trade failed" toast. No items lost.

8. Trade history is logged server-side for anti-fraud auditing.

While in trading mode, the player cannot modify their own backpack outside the trade UI (other UIs are disabled), can't be teleported, can't accept other trade requests.

Anti-duping protection

Critical to get right. Trading economies die fast if duping is possible. Required:

• All item moves go through the server with idempotent operation IDs.

• Both players' offers are committed atomically server-side — no intermediate state where one player has both sets of items.

• If the server crashes mid-trade, on restart the trade is rolled back (both players keep their original items).

• Inventory mutations during a trade are blocked (can't drop the item you're offering).

• Rate limit: max 1 trade every 10 seconds per player.

System: Player Stalls (rented shops)

At higher reputation (default ≥ 1000), players can rent a stall in the Trading Post and list items for sale at fixed prices or auction. This is the long-tail player-economy feature.

Stall mechanics

1. Player walks to the Stall Manager NPC. Pays rental fee (e.g., 1000 coins/day).
2. Stall is assigned (a numbered Model in the Trading Post). Player can decorate it (place a sign, banner — uses the decoration system).
3. Player opens stall management UI. Lists items in inventory with: "Add to stall", quantity, price per unit. Toggle: fixed-price or auction (with min bid + duration).
4. Other players approaching the stall see the listings. Tapping a listing → buy prompt → completes purchase (coins debited, item added to buyer's inventory; coins credited to stall owner; stall inventory decreases).
5. Owner gets notification of sale (replicates next time they log in).
6. Stalls auto-renew daily if owner has the rental fee in their bank; otherwise listings return to owner's inventory and stall is released.

All transactions are server-side. Fees taken automatically. Audit log of all stall sales for anti-fraud.

System: House / hut customization + plot visiting

Each player owns a hut. The hut is a small interior space (instanced — separate Place or RoomService). Decoration system lets players place items to customize. Other players can visit if invited or if the hut is set to public.

Hut basics

• On first spawn, player is given a basic hut (defined in Workspace as a tagged Model). Can be expanded via gamepass (House Upgrade).

• Hut interior contains: a personal cauldron, a personal storage chest (extends inventory), spirit display shelf, decoration anchors.

• Decoration items (purchased from Decoration Merchant or earned from quests) can be placed in the hut via a placement UI: pick item → ghost preview follows cursor → tap to place. Long-press to remove.

• Player can change wall color, floor texture, lighting via Hut Settings UI (cycling through unlocked options).

Plot visiting

1. Player taps "Visit plot" on another player's character menu OR opens the friend list and taps a friend.
2. If destination player's hut is set to Public OR the visiting player is on their friend list with hut access enabled — proceed. Otherwise show "This player's hut is private."

3. TeleportService:Teleport to the destination player's hut Place.

4. While visiting: visitor cannot modify the host's hut, plots, or inventory. Can only walk around and observe.

5. Host's hut shows host's plots, spirits, decorations.

6. Visitor can tap a friendly action button to leave a guest message in the host's guestbook (a hut interior item).

System: Player progression + reputation

No traditional XP/level system. Progression is reputation with NPCs + species discovery + wealth + collection completeness.

Reputation system:

- Each NPC has a reputation score with each player (0 to a high cap, default 5000).
- Completing quests for an NPC increases their reputation. Selling them mushrooms (preferred categories) gives small reputation. Buying from them gives small reputation.
- Reputation gates: certain quests, biome unlocks, advanced shop items require reputation thresholds with specific NPCs.
- Total reputation across all NPCs is the player's overall "renown" — used as a soft progression indicator. Displayed in the player profile.

Other tracked stats (already partially in PlayerData):

- totalHarvested, totalPlanted, totalSold, totalCoinsEarned, totalBrewed
- speciesDiscovered set, potionsDiscovered set
- spiritsCollected set
- biomesUnlocked set
- questsCompleted set
- playtimeSeconds (incremented on a server tick while player is in-server)

System: Monetization (Robux)

Critical: never sell power, rarity, or trading advantages. Sell time and cosmetics. Pay-to-win kills trading economies — the Adopt Me lesson.

Game Passes (one-time purchases)

- Bigger Plot — adds 4 more plots to the player's owned set (max 16 total via stacking). Convenience.

- Auto-Harvester — passively gathers ripe mushrooms on the player's plots while offline (up to a per-day cap). Quality of life.
- Recipe Journal Plus — better organization for the Brewing Journal: tags, notes, sort by ingredient.
- Inventory Expansion — adds 4 more rows of slots per category in the backpack.
- Bigger Spirit Roster — increases active spirit slot count from 3 to 6.
- Recyclable Substrate — substrates are not consumed on harvest; reusable.
- Cosmetic outfit packs — multiple, each unlocks a set of character outfits.
- House Upgrade — bigger hut interior with more decoration anchors.

Developer Products (consumables)

- Coin Pack — small / medium / large purchasable coin bundles. Carefully scaled so they accelerate but never skip progression.
- Speed-Up Potion — instantly ripens all of the player's growing plots. Limit: once per real hour.
- Cosmetic Spirit Skin — alternative visual for an owned spirit (purely visual, doesn't change effect).
- Decoration Items — premium hut decorations.

Purchase flow

1. Player opens the Shop UI from the HUD. Tabs: Game Passes / Robux Items / Coin Shop / Cosmetics.
2. Each item shows name, description, price (Robux or coins), preview thumbnail, Buy button.
3. On tap: trigger MarketplaceService:PromptGamePassPurchase or PromptProductPurchase as appropriate.
4. On purchase confirmed (MarketplaceService.PromptGamePassPurchaseFinished / PromptProductPurchaseFinished events): server applies the effect (add coins, grant gamepass attribute, etc.) and notifies the client.

Game pass ownership is checked on player join (MarketplaceService:UserOwnsGamePassAsync) and cached in PlayerData. Players who buy gamepasses mid-session get them applied immediately.

System: HUD + UI scripting

User supplies UI visual designs (mockups, button textures, panel art, color palettes, fonts). Scripter wires them up: handles button events, opens/closes panels, binds data, animates transitions, handles mobile vs PC input layouts.

HUD elements (always-visible)

- Top-left: Stats card (Coins + Inventory total).
- Below stats: row of action buttons (Backpack, Pokedex, Brewing Journal, Quests, Trading, Settings, Shop).
- Below buttons: Active Effects strip (one chip per active potion effect, color-coded with countdown).
- Top-right: Tracked Quest pinned widget (objective summary).
- Bottom-center (mobile): mobile control overlay (jump, action buttons).
- Bottom-right: chat bubble icon (opens chat).
- Top-center: notification toasts (transient — quest progress, item gains, achievements). Stack vertically.

Modal panels (one open at a time)

- Backpack
- Pokedex
- Brewing Journal
- Quest Journal
- Shop (per-NPC variant)
- Trading UI
- Settings
- Player Profile

All modals follow the same shell: semi-transparent overlay (taps outside to close), centered card with a close X button, scrollable content area, animated open/close (0.2s ease).

Mobile considerations

- All interactive elements ≥ 44pt tap targets.
- Bottom-heavy layouts where possible (single-thumb operability).
- Pickers (planting, brewing) use scrollable lists, not nested menus.
- Test every screen on a 6.1" phone before shipping. Mobile is the primary platform.

System: Camera scripting

Default Roblox third-person camera, with these enhancements:

- Smooth zoom in/out on scroll (PC) or pinch (mobile). Min zoom: 8 studs (close shoulder view). Max zoom: 30 studs (overview).
- Cinematic camera moments — when discovering a new species, briefly orbit the camera around the harvest point for ~2 seconds before returning to player control.
- When opening certain modals (Pokedex, Trading), gently zoom out and slightly desaturate the world (ColorCorrectionEffect tween).
- On lunar event start (Forest Mother spawn, etc.), camera does a brief pan to the event location across all clients in the server.
- Camera shake on rare events (legendary spirit summon, server-wide lunar event) — small amplitude, 1-second duration.

System: Animation scripting

User supplies all animation assets (rigged characters, NPC idles, mushroom growth animations, etc.). Scripter wires up Animation/AnimationController instances and triggers playback on game events.

Animations to wire up:

- Player character: idle, walk, run, harvest, plant, brew, sell, jump, sit.
- NPCs: idle, talking gesture, ambient action (witch stirring her cauldron, hermit reading a book, etc.).
- Mushroom growth: each plot's GrowingVisual contains a sprout that scales up over time. Lerp the scale from 0.1 → 1.0 over the growth duration on the client (server only sets the start time).
- Plot ripen: trigger a brief glow + scale-up wobble on the cap when the plot becomes Ripe.
- Harvest pickup: small upward float + fade for the harvested mushroom (visual feedback).
- Spirit wandering: cycle idle / walk animations as the spirit moves between waypoints.
- UI panel open/close: tween size + transparency 0.2s ease.
- Toast notifications: slide in from top, hold for N seconds, fade out.

System: Sound scripting

User supplies all audio assets (ambient music tracks, SFX). Scripter loads them via SoundService, plays them on the right triggers, manages volume + spatial audio.

Audio to wire up:

- Per-biome ambient music — loops on biome entry. Crossfade 2 seconds when changing biomes.
- Per-season ambient layer — gentle layer over the biome music (birdsong in spring, wind in winter).
- Pond water sound — spatial 3D audio at the pond center. Audible within ~30 studs.
- Wild forest ambience — birdsong, distant streams, wind through leaves. Looped, positional.
- SFX (one-shots): harvest puff, plant thud, brew bubble, brew complete chime, brew fizzle, sell coin sound, level up chime, discovery chime, quest accept, quest complete.
- UI SFX: button press, panel open, panel close, toast notification.
- Spirit sound: when a spirit appears, brief magical chime.
- Lunar event audio: ambient drone increases over 10 seconds before the event triggers, then a distinctive musical sting at the moment of trigger.

All audio respects a master volume slider in Settings. SFX, Music, and Ambient have separate sub-sliders. Mute-when-tabbed-out enabled by default.

System: VFX scripting

User supplies particle definitions (ParticleEmitter properties + texture asset IDs). Scripter creates emitter Instances, parents them to the right places, triggers them on events.

VFX to wire up:

- Spore puff: brief particle burst on harvest (both wild and plot harvest).
- Brew bubbles: continuous emitter on the cauldron Liquid Part.
- Discovery glow: when a player discovers a new species or recipe, brief gold particle burst around their character.
- Spirit summon: rising motes + brief light flash when a spirit spawns.
- Lunar event: large central beam of light at the event location, visible from across the biome.
- Seasonal: snow particles in winter (per biome), leaf-fall in autumn, rain particles when weather is rainy, fog particles in foggy weather.
- Ambient: fireflies in the wild mushroom field at evening, glowing pollen drifting through Glimmerwood.
- Cosmetic potion effect: glow trail behind the player character while a cosmeticGlow potion is active (color depends on potion).

System: Save data + DataStore

Existing PlayerData.lua handles save schema v2. Extend for the full feature set.

Full save schema (v3 target):

- schemaVersion (number)
- coins (number)
- inventory: { [itemId] = count }
- potions: { [potionId] = count }
- spirits: { activeRoster, allOwned }
- decorations: { unlocked, placed }
- stats: { totalHarvested, totalPlanted, ..., speciesDiscovered, potionsDiscovered, spiritsCollected, biomesUnlocked, questsCompleted }
- reputation: { [npcId] = score }
- activeQuests: { [questId] = { progress } }
- completedQuests: { [questId] = completedAt }
- gamepassesOwned: { [gamePassId] = true }
- settings: { masterVolume, musicVolume, sfxVolume, hutAccessLevel, etc. }

Save mechanics:

- Autosave every 60 seconds per player.
- Save on player leaving + on game shutdown (BindToClose).
- Migration on schema version bump (v2 → v3 requires backfilling new fields).
- Pcalled DataStore calls — graceful degradation when DataStore unavailable (gameplay still works in unpublished places).
- Anti-rollback: on suspicious load (load fails or returns nil for a known player), refuse to save until next clean load — prevents wiping a player's data with a default.

Migrate to ProfileService (or similar session-locking library) for production. The current DataStore wrapper is sufficient for Pre-Alpha but doesn't handle session locking, retries, or duped-data prevention robustly. ProfileService is the standard Roblox library for this.

System: Anti-exploit + security

Critical for trading economies. Bake in from day one — retrofitting is much harder.

- All gameplay state mutations go through server. Clients never write to PlayerData directly.
- All remote handlers validate every input: type-check arguments, range-check numbers, validate referenced ids exist.
- Rate limit per-player on every remote: max 5 calls per second per remote, configurable per remote in Constants.RATE_LIMITS.

- Atomic operations: trades commit both sides atomically; brewing deducts ingredients before potion is granted; etc. No partial-state windows.
- Idempotent operation IDs on critical operations (trades, purchases) so a duplicate request from a buggy client doesn't double-execute.
- Audit log: every coin/item movement is logged server-side with timestamp + actors. Stored in a separate DataStore for fraud investigation.
- Inventory cap enforcement: server rejects adds that would exceed slot or stack limits.
- Coin cap: hard ceiling at, say, 1 trillion to prevent integer overflow exploits.
- Sanity checks: if a player gains > 1M coins or > 1000 items in a single tick, log + flag for review.

System: LiveOps infrastructure

Static games die. Need cadenced events from launch.

Event types:

- Daily login bonus — log in once per day to get a small reward (escalating bonus over consecutive days).
- Daily quest from the Forest Witch — random tutorial quest. Refreshes at midnight UTC.
- Weekly events — limited-time mushroom species in rotation. One biome gets a weekly event with bonus spawn rates of a specific species.
- Monthly major events — mythic spawns, new spirit released, new recipe ingredients introduced.
- Seasonal — every 3 months, new biome introduced, major story beat, exclusive cosmetic line.

Infrastructure needed:

- Server-side event scheduler that reads from a config (HttpService → external endpoint, or MessagingService → admin-pushed updates).
- Per-event flag in PlayerData: hasClaimedDailyLogin (timestamp), eventParticipation (set of event ids).
- Event UI: HUD button that opens an event-info panel with current event details, leaderboard, claimed rewards.
- Push-notification analog: when an event starts, all players in-server get a top-screen toast and a chat-channel message.

System: Friends + chat

Roblox provides default chat. We extend it with:

- Custom chat channels: /faction (for trading-post-rented stall owners), /trade (for active traders), /lfg (for expedition recruiting).
- /whisper [player] [message] for private DMs (tracked by chat history server-side for moderation).
- Friend list integration: tap a friend's name in the friend list to see their current biome + their hut access level. Buttons: Visit Plot, Send Trade Request, Whisper.
- Social tab in the HUD: friends online, recent trade partners, party members.

Summary of the work

Building all of the above takes a Roblox-experienced scripter on the order of months (not weeks). Suggested phasing:

- Phase 1 (Alpha): finish brewing depth (more recipes, more effect kinds), implement Forest Spirits, second biome (Misty Hollow).
- Phase 2 (Closed Beta): trading post + player stalls, co-op foraging expeditions, full quest system, 30+ species.
- Phase 3 (Soft Launch): polish, monetization, full LiveOps infrastructure, mobile optimization pass.
- Phase 4 (Global Launch): remaining biomes, full 60-species catalog, event cadence locked in.

Pre-Alpha to Soft Launch: realistic 6–8 months of full-time scripting work for one experienced developer. Faster with a team.

Notes on what I'm supplying vs scripter-built

I will provide:

- All 3D models (mushrooms, NPCs, props, hut interiors, biome assets).
- All UI visual designs and assets (mockups, button textures, panel art, color palettes, fonts).
- All character and NPC animations (rigged + animated separately).
- All audio assets (ambient music tracks, SFX, voice lines if any).
- All VFX assets (particle textures, emitter parameter specs).

Scripter builds:

- All gameplay code (server + client logic, validation, persistence, networking).
- All UI behavior wiring (button handlers, panel state, data binding, animations between states).

- Animation playback scripting (Animator instances, trigger conditions).
- Sound playback scripting (SoundService, spatial audio, music transitions).
- Camera scripting (zoom, cinematic moments, modal-open transitions).
- VFX wiring (instantiating emitters, triggering on events).
- Anti-exploit protections, server-side validation, audit logging.
- DataStore + save migration logic.
- LiveOps infrastructure for event delivery.
- Tests for new systems following the existing TestKit pattern.

Codebase rules

- Server-authoritative everything. Clients send intents; server validates + replies. Never trust client input.
- Tunables in Constants.lua. Anything balance-affecting (costs, durations, multipliers, drop rates) belongs in src/ReplicatedStorage/Shared/Constants.lua.
- Species ids never change once shipped. They're the save-data key.
- No pay-to-win. Robux can unlock time and cosmetics, never rare items, recipes, or trading advantages.
- Hand-built escape valves exist for procedurally-built world pieces. See MapSetup.lua and TerrainBuilder.lua docstrings.
- Add tests for new gameplay-affecting code. Pattern: pure-function modules get TestKit specs in src/ServerScriptService/Tests/.